Grid Shortest Path

Graphs, Matrix as Graph, BFS Shortest Path · Mentor-Led Lab (Student Edition)

Developed by TalenciaGlobal

Difficulty ★★★ Advanced

Problem

On a grid (0 = open, 1 = wall) find the shortest number of moves from the top-left to the bottom-right, moving through open cells in four directions. You are given starter code with two functions. bfs_distance is almost complete but has a bug, and can_reach is an empty stub. Fix the bug and implement the stub.

What to Compute

- Distance: the shortest number of moves from (0,0) to (R-1,C-1), or -1 if there is no path.
- Reachable: Yes if the exit is reachable, otherwise No.

Input / Output Format

Input: Line 1: R C. Next R lines: C integers each (0 = open, 1 = wall).

Output: Two lines: Distance and Reachable.

Examples

Example 1

Input: 3 3 0 0 1 1 0 1 1 0 0

Output:

```
Distance: 4
Reachable: Yes
```

Explanation: The only route (0,0)-(0,1)-(1,1)-(2,1)-(2,2) is four moves, so the exit is reachable in distance 4.

Example 2

Input: 2 2 0 0 0 0

Output:

```
Distance: 2
Reachable: Yes
```

Explanation: An open 2x2 grid reaches the opposite corner in two moves.

Constraints

- Move only through open (0) cells, four directions.
- The start cell is distance 0.
- If the start or exit is blocked, or no path exists, the distance is -1.

Your Task

- Task 1 - Fix the bug. bfs_distance is flagged with a comment, but the bug's location is not given. Find the single bug so the start cell is at distance 0 (the answer counts moves, i.e. edges).
- Task 2 - Implement can_reach from scratch: whether the exit is reachable from the start.

Starter Code

Begin from the code below. One function carries a single bug (flagged by a comment, with no hint to its location); the other is an empty stub you must implement.

Python

```
import sys
from collections import deque

def read_grid(data):
    idx = 0
    r = int(data[idx]); c = int(data[idx + 1]); idx += 2
    grid = []
    for _ in range(r):
        row = []
        for _ in range(c):
            row.append(int(data[idx])); idx += 1
        grid.append(row)
    return r, c, grid

# BUG: this function contains a single bug. The distance is measured in STEPS (edges),
# so the start cell must be at distance 0.
def bfs_distance(r, c, grid):
    if grid[0][0] == 1 or grid[r - 1][c - 1] == 1:
        return -1
    dist = [[-1] * c for _ in range(r)]
    dist[0][0] = 1
    q = deque([(0, 0)])
    while q:
        x, y = q.popleft()
        for dx, dy in ((-1, 0), (1, 0), (0, -1), (0, 1)):
            nx, ny = x + dx, y + dy
            if 0 <= nx < r and 0 <= ny < c and grid[nx][ny] == 0 and dist[nx][ny] == -1:
                dist[nx][ny] = dist[x][y] + 1
                q.append((nx, ny))
    return dist[r - 1][c - 1]
```

```
# TODO: implement this. It must return True if the exit is reachable from the start.
# Currently a stub.
def can_reach(r, c, grid):
    return False

def main():
    data = sys.stdin.read().split()
    r, c, grid = read_grid(data)
    d = bfs_distance(r, c, grid)
    print("Distance: " + str(d))
    print("Reachable: " + ("Yes" if can_reach(r, c, grid) else "No"))

main()
```

C++

```
// Graph Lab 5 - Grid Shortest Path (STARTER)
#include <iostream>
#include <vector>
#include <queue>
using namespace std;

int r, c;

// BUG: this function contains a single bug. The distance is measured in STEPS (edges),
// so the start cell must be at distance 0.
int bfsDistance(const vector<vector<int>>& grid) {
    if (grid[0][0] == 1 || grid[r - 1][c - 1] == 1) {
        return -1;
    }
    vector<vector<int>> dist(r, vector<int>(c, -1));
    dist[0][0] = 1;
    queue<pair<int,int>> q;
    q.push(make_pair(0, 0));
    int dx[] = {-1, 1, 0, 0};
    int dy[] = {0, 0, -1, 1};
    while (!q.empty()) {
        pair<int,int> cur = q.front();
        q.pop();
        for (int d = 0; d < 4; d++) {
            int nx = cur.first + dx[d];
            int ny = cur.second + dy[d];
            if (nx >= 0 && nx < r && ny >= 0 && ny < c && grid[nx][ny] == 0 && dist[nx][ny] ==
-1) {
                dist[nx][ny] = dist[cur.first][cur.second] + 1;
                q.push(make_pair(nx, ny));
            }
        }
    }
    return dist[r - 1][c - 1];
}

// TODO: implement this. It must return true if the exit is reachable from the start.
// Currently a stub.
bool canReach(const vector<vector<int>>& grid) {
    return false;
```

```
}

int main() {
    cin >> r >> c;
    vector<vector<int>> grid(r, vector<int>(c));
    for (int i = 0; i < r; i++) {
        for (int j = 0; j < c; j++) {
            cin >> grid[i][j];
        }
    }
    int d = bfsDistance(grid);
    cout << "Distance: " << d << "\n";
    cout << "Reachable: " << (canReach(grid) ? "Yes" : "No") << "\n";
    return 0;
}
```

Java

```
// Graph Lab 5 - Grid Shortest Path (STARTER)
import java.util.*;

public class Main {
    static int r, c;

    // BUG: this function contains a single bug. The distance is measured in STEPS (edges),
    // so the start cell must be at distance 0.
    static int bfsDistance(int[][] grid) {
        if (grid[0][0] == 1 || grid[r - 1][c - 1] == 1) {
            return -1;
        }
        int[][] dist = new int[r][c];
        for (int[] row : dist) {
            Arrays.fill(row, -1);
        }
        dist[0][0] = 1;
        Queue<int[]> q = new LinkedList<>();
        q.add(new int[] {0, 0});
        int[] dx = {-1, 1, 0, 0};
        int[] dy = {0, 0, -1, 1};
        while (!q.isEmpty()) {
            int[] cur = q.poll();
            for (int d = 0; d < 4; d++) {
                int nx = cur[0] + dx[d];
                int ny = cur[1] + dy[d];
                if (nx >= 0 && nx < r && ny >= 0 && ny < c && grid[nx][ny] == 0 && dist[nx][ny] ==
-1) {
                    dist[nx][ny] = dist[cur[0]][cur[1]] + 1;
                    q.add(new int[] {nx, ny});
                }
            }
        }
        return dist[r - 1][c - 1];
    }

    // TODO: implement this. It must return true if the exit is reachable from the start.
    // Currently a stub.
    static boolean canReach(int[][] grid) {
```

```
        return false;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        r = sc.nextInt();
        c = sc.nextInt();
        int[][] grid = new int[r][c];
        for (int i = 0; i < r; i++) {
            for (int j = 0; j < c; j++) {
                grid[i][j] = sc.nextInt();
            }
        }
        int d = bfsDistance(grid);
        System.out.println("Distance: " + d);
        System.out.println("Reachable: " + (canReach(grid) ? "Yes" : "No"));
    }
}
```

Sample Run

Sample Input

```
3 3
0 0 1
1 0 1
1 0 0
```

Sample Output

```
Distance: 4
Reachable: Yes
```